

Probability of Spontaneous Self-Replication in a 6502 Cellular Automaton

March 2026

Abstract

We compute the probability that a uniformly random 1024-byte program in a 6502-based cellular automaton constitutes a functional self-replicator. The strict minimal replicator requires 9 specific bytes, giving a base probability of 2^{-72} . We systematically relax the requirements by cataloging functionally equivalent instruction sequences: alternative target cells, alternative loop mechanisms, index register variants, and NOP-insertion tolerance. These relaxations reduce the effective information content to approximately 43 bits, yielding a probability of roughly 2^{-43} . We discuss implications for “primordial soup” experiments on random-initialized boards.

Contents

1	System Model	2
2	The Minimal Replicator	2
2.1	JMP variant (9 bytes)	2
2.2	BRK variant (7 bytes)	3
2.3	Branch variant (8 bytes)	3
3	Base Probability	3
4	Relaxation Analysis: Byte-by-Byte Constraints	4
4.1	Byte 0: Load opcode	4
4.2	Byte 1: Zero-page base address	5
4.3	Byte 2: Store opcode	5
4.4	Byte 3: Low byte of destination address	5
4.5	Byte 4: High byte of destination address	5
4.6	Byte 5: Increment/decrement opcode	6
4.7	Bytes 6+: Loop instruction	6
4.7.1	JMP variant (bytes 6–8)	6
4.7.2	Branch variant (bytes 6–7)	6
4.7.3	BRK variant (bytes 6+)	7
5	Summary of Byte Constraints	7
5.1	JMP variant (9 bytes, random initialization)	8
5.2	Branch variant (8 bytes, random initialization)	8
5.3	BRK variant (random initialization)	8
5.4	Combined probability (union over all variants)	9
6	NOP Insertion Tolerance	9
6.1	Setup	9
6.2	Counting NOP opcodes	9
6.3	Generating function for NOP insertions	10
6.4	Multi-byte NOP instructions	11

7	Additional Functional Equivalences	13
7.1	Starting position within the cell	13
7.2	Alternative copy directions	13
7.3	Copy to different cells	13
7.4	Partial copies and multi-quantum replication	13
7.5	Self-modifying or longer replicators	13
8	Effective Information Content	13
8.1	BRK variant, random initialization (most favorable)	13
8.2	Branch variant, random initialization, generous opcode count	14
8.3	Union over all variants with NOP correction	14
9	Primordial Soup: Probability of Spontaneous Emergence	14
9.1	Expected number of replicators on a random board	14
9.2	Threshold board size	15
9.3	Memory requirements	15
9.4	Comparison with biological estimates	15
10	Sensitivity Analysis	15
10.1	Undocumented NOP counts	15
10.2	Functional but imperfect replicators	16
10.3	Replicator as a substring	16
11	Dominance Time and Lottery Economics	16
11.1	Replicator dominance time	16
11.2	Throughput: boards tested per unit time	16
11.3	Lottery ticket probability	17
12	Conclusion	18

1 System Model

We consider the 6502life cellular automaton [1]. Each cell contains $M = 1024$ bytes of RAM. Execution begins at address \$0000 after each scheduler interrupt. The cell’s own memory occupies addresses \$0000–\$03FF (cell 0 in the memory-mapped neighborhood). The 48 neighbor cells (in a 7×7 grid minus the origin) are mapped at addresses \$0400–\$BFFF, with cell c occupying addresses $\$0400 \cdot c$ through $\$0400 \cdot c + \$03FF$ for $c = 1, \dots, 48$.

The scheduler randomly selects a cell, assigns it a random orientation (one of four 90° rotations), and resumes execution at the cell’s program counter. After a Poisson-distributed number of cycles, the scheduler interrupts the cell, saves its registers to fixed locations in page 0, and moves to the next cell.

Definition 1 (Byte-level self-replicator). *A program starting at address \$0000 is a self-replicator if, when executed from a state with the X register initialized to some value x_0 , it eventually writes a complete copy of its own page 0 (bytes \$00–\$FF) to the page 0 of some neighbor cell c ($1 \leq c \leq 48$), such that the copy is byte-identical to the original at least at all positions occupied by the replicator’s own code.*

Remark 1. *This definition is deliberately permissive: the replicator need only preserve its own code bytes in the copy. Other bytes in page 0 (stack, data areas) may differ. In practice, the minimal replicators we analyze copy the entire 256-byte page 0, which is stronger than necessary.*

2 The Minimal Replicator

2.1 JMP variant (9 bytes)

The shortest known self-replicator that uses an explicit jump is:

```
LDA $00,X ; B5 00 load byte X from own page 0
STA $0400,X ; 9D 00 04 store to neighbor cell 1, page 0
INX ; E8 increment index
JMP $0000 ; 4C 00 00 loop
```

The byte sequence is: B5 00 9D 00 04 E8 4C 00 00.

This program copies byte X from its own page 0 to offset X in cell 1's page 0, increments X , and loops. Since X is an 8-bit register that wraps from $\$FF$ to $\$00$, after 256 iterations every byte of page 0 has been copied, including the replicator's own code. The copy is therefore byte-identical to the original.

2.2 BRK variant (7 bytes)

The opcode BRK ($\$00$) with operand $\$00$ resets the program counter to $\$0000$ and yields to the scheduler. If the cell has enough remaining cycles, execution resumes at $\$0000$. In zero-initialized RAM, the byte following INX is already $\$00$, so:

```
LDA $00,X ; B5 00
STA $0400,X ; 9D 00 04
INX ; E8
BRK 0 ; 00 00 (second 00 is BRK operand, already zero)
```

Byte sequence: B5 00 9D 00 04 E8 00. The trailing $\$00$ (BRK operand) is “free” in zero-initialized RAM. The 6502 BRK instruction always reads one operand byte; here it reads byte 7 which is zero.

Remark 2. *The BRK variant has a subtlety: BRK triggers the scheduler, which re-randomizes the cell's orientation and may reassign the neighbor mapping. However, the copy loop is stateless (X wraps modulo 256), so after each scheduler re-entry the copy continues from whatever X value was saved. The destination cell changes between interrupts, so the replicator distributes partial copies across multiple neighbors. For complete self-replication, the replicator must execute 256 consecutive iterations to a single target without interruption. The expected number of cycles for 256 iterations of the 4-instruction loop is approximately $256 \times 10 = 2560$ cycles. Whether this completes before the Poisson-distributed interrupt depends on the scheduler's mean quantum.*

2.3 Branch variant (8 bytes)

A two-byte branch instruction can replace the three-byte JMP:

```
LDA $00,X ; B5 00
STA $0400,X ; 9D 00 04
INX ; E8
BNE $F8 ; D0 F8 branch back to offset 0 if Z=0
```

Byte sequence: B5 00 9D 00 04 E8 D0 F8 (8 bytes).

The branch offset is computed as: from byte 6, BNE is 2 bytes, so $PC = 8$ after fetch. Target is byte 0. Offset = $0 - 8 = -8$, encoded as the two's complement byte $\$F8$.

After INX, the zero flag Z is clear unless X wrapped to zero. So the branch is taken for 255 of every 256 iterations. On the 256th iteration ($X = 0$), the branch falls through, and execution continues with whatever bytes follow. In zero-initialized RAM, the next byte is $\$00 = \text{BRK}$, which resets PC to 0 and resumes. So the branch variant is a complete replicator, with a brief one-time BRK detour every 256 iterations.

3 Base Probability

Proposition 1 (Strict base probability). *If all 1024 bytes are drawn independently and uniformly from $\{0, 1, \dots, 255\}$, the probability that the first N bytes match a specific N -byte replicator pattern is*

$$P_{\text{base}} = 256^{-N} = 2^{-8N}.$$

Variant	N (bytes)	P_{base}
JMP	9	2^{-72}
Branch	8	2^{-64}
BRK	7	2^{-56}
BRK (free trailing zero)	6	2^{-48}

For the BRK variant, if we only count the 6 “non-free” bytes (assuming the 7th byte is zero in the initial state), then $P_{\text{base}} = 2^{-48}$.

4 Relaxation Analysis: Byte-by-Byte Constraints

We now analyze each byte position to determine how many of the 256 possible values yield a functionally equivalent replicator.

4.1 Byte 0: Load opcode

The load instruction must read a byte from the cell’s own page 0 using an index register. The valid opcodes are:

Opcode	Mnemonic	Addressing Mode	Register used
\$B5	LDA zp, X	zero page, X-indexed	X
\$B6	LDX zp, Y	zero page, Y-indexed	$Y^\dagger$

† LDX zp, Y loads into the X register, but is indexed by Y. This requires the corresponding store and increment to also use Y (or X), which we will account for in those positions.

If we use LDA zp, X , then the store must be STA abs, X and the increment must be INX. If we use LDX zp, Y , then the store must be a different instruction (since there is no STX abs, Y on the 6502). We could use LDA zp, Y (\$B6... wait, \$B6 is LDX zp, Y).

Let us be more systematic. The 6502 zero-page indexed addressing modes for loads are:

- \$B5: LDA zp, X — loads A, indexed by X
- \$B6: LDX zp, Y — loads X, indexed by Y
- \$B4: LDY zp, X — loads Y, indexed by X

For a replicator, we need a load–store pair that reads from page 0 and writes to an absolute indexed address. The absolute indexed stores are:

- \$9D: STA abs, X — stores A, indexed by X
- \$99: STA abs, Y — stores A, indexed by Y

There is no STX abs, Y or STY abs, X in absolute indexed mode on the 6502.

Therefore the viable load–store–increment combinations are:

Load	Store	Increment	Index register
LDA zp, X (\$B5)	STA abs, X (\$9D)	INX (\$E8) or DEX (\$CA)	X
LDA zp, Y^*	STA abs, Y (\$99)	INY (\$C8) or DEY (\$88)	Y

*There is no official LDA zp, Y instruction on the 6502. The zero-page Y-indexed addressing mode exists only for LDX and STX. To load A from a Y-indexed zero-page address, one would need indirect indexed addressing (LDA (zp), Y, opcode \$B1), which requires a pointer in zero page and targets a different address range.

Revised analysis: The only working combination using official opcodes and absolute indexed addressing is the X-register variant:

- Load: \$B5 (LDA zp,X) — 1 valid value
- Store: \$9D (STA abs,X) — 1 valid value
- Increment: \$E8 (INX) or \$CA (DEX) — 2 valid values

However, we can also construct a replicator using indirect indexed addressing:

```
LDA ($00),Y ; B1 00 load via pointer at $00,$01
STA ($02),Y ; 91 02 store via pointer at $02,$03
INY ; C8
BNE $FA ; D0 FA branch back
```

This requires the zero-page pointers to be set up correctly: $\$00, \$01 = \$00, \00 (pointer to \$0000) and $\$02, \$03 = \$00, \04 (pointer to \$0400). But the replicator’s own code occupies bytes 0–7, which conflicts with the required pointer values at bytes 0–3. This variant is therefore **not self-consistent** and we exclude it.

Conclusion for byte 0: Exactly 1 valid value (\$B5). Constraint: 8 bits.

4.2 Byte 1: Zero-page base address

The operand of LDA zp,X specifies the base address b . The effective address is $(b + X) \bmod 256$. Source byte read when index register equals X is memory byte $(b + X) \bmod 256$.

The store instruction STA \$XX00,X writes to absolute address $\$XX00 + X$, i.e., offset X in the target cell’s page 0.

Therefore, destination byte at offset k receives source byte $(b + k) \bmod 256$. This is a *rotation* of source page 0 by b positions.

For the copy to be self-replicating, the destination’s bytes at the replicator code positions must match the source’s bytes at those same positions. Let the replicator occupy bytes $0, 1, \dots, L - 1$ and let bytes $L, L + 1, \dots, 255$ be “background” (zero in a fresh cell).

Destination byte $k = \text{source byte } (k + b) \bmod 256$. For $k = 0, 1, \dots, L - 1$: we need source byte $(k + b) \bmod 256 = \text{source byte } k$ (the replicator’s k -th byte).

If $b = 0$, this is trivially satisfied.

If $b \neq 0$, then $(k + b) \bmod 256 \neq k$, and source byte $(k + b) \bmod 256$ is generically a background byte (zero), not the replicator byte at position k . The only exception would be if the replicator code happened to be periodic with period dividing b , which our replicator is not (it contains the specific bytes B5 00 9D 00 04 E8 ..., which are not periodic).

Conclusion for byte 1: Must be \$00. Constraint: 8 bits.

4.3 Byte 2: Store opcode

Must be \$9D (STA abs,X). Constraint: 8 bits.

4.4 Byte 3: Low byte of destination address

The store instruction STA \$HHLL,X writes to address $\$HHLL + X$. The low byte LL determines the base offset within the target cell’s page.

By the same rotation argument as for byte 1: destination byte at offset k receives the value of the source byte loaded when $X = k$. The address written is $\$HH \cdot 256 + LL + X$. Cell c occupies addresses $1024c$ through $1024c + 1023$. The offset within cell c is $(LL + X) \bmod 1024$.

For page 0 of cell c (offsets 0–255): the write hits offset $(LL + X) \bmod 256$ of page 0, provided \$HH selects page 0 of cell c (which requires $\$HH = 4c$). If $LL = 0$, write offset equals X , and the copy is unrotated. If $LL \neq 0$, the copy is rotated by LL , and the same analysis as byte 1 applies: the replicator is not self-consistent.

Conclusion for byte 3: Must be \$00. Constraint: 8 bits.

4.5 Byte 4: High byte of destination address

The high byte \$HH must equal $4c$ for some valid neighbor cell $c \in \{1, 2, \dots, 48\}$, so that the store targets page 0 of a neighbor cell.

The valid values are $\{4, 8, 12, \dots, 192\}$, i.e., the set $\{4c : 1 \leq c \leq 48\}$. These are 48 distinct values, all multiples of 4 in the range $[4, 192]$.

The probability that a uniform random byte is one of these 48 values is $48/256 = 3/16$.

Conclusion for byte 4: 48 valid values out of 256. Constraint: $\log_2(256/48) = \log_2(16/3) \approx 2.415$ bits.

4.6 Byte 5: Increment/decrement opcode

The index register X must change by ± 1 each iteration so that all 256 byte offsets are visited.

- \$E8: INX — increments X. Copies bytes in order $0, 1, 2, \dots, 255$ (starting from initial X).
- \$CA: DEX — decrements X. Copies bytes in order $0, 255, 254, \dots, 1$. All 256 bytes are still copied.

No other single-byte instruction changes X by exactly ± 1 without side effects that would corrupt the replicator.

Valid values: 2 out of 256. Constraint: $\log_2(256/2) = 7$ bits.

4.7 Bytes 6+: Loop instruction

We analyze three loop mechanisms.

4.7.1 JMP variant (bytes 6–8)

JMP \$0000 is encoded as 4C 00 00 (3 bytes). The operand must be exactly \$0000 to return to the start. Constraint: $8 + 8 + 8 = 24$ bits (3 bytes).

No alternative: JMP is the only 6502 instruction that performs an unconditional absolute jump. (JMP (indirect) at \$6C requires a pointer, adding complexity.)

4.7.2 Branch variant (bytes 6–7)

A conditional branch instruction is 2 bytes: opcode plus signed offset. The offset must be \$F8 ($= -8$ signed) to branch back to byte 0 from byte 6 (since PC after fetch = $6 + 2 = 8$, and $8 + (-8) = 0$).

The branch is taken if its condition is met. After INX (or DEX), the processor flags are:

- $Z = 1$ iff $X = 0$ (wrapped). Otherwise $Z = 0$.
- $N = 1$ iff bit 7 of X is set (i.e., $X \in [128, 255]$). Otherwise $N = 0$.
- C is *unchanged* by INX/DEX.
- V is *unchanged*.

The branches and their reliability:

Opcode	Mnemonic	Condition	Iterations taken (of 256)
\$D0	BNE	$Z = 0$	255
\$F0	BEQ	$Z = 1$	1
\$10	BPL	$N = 0$	128
\$30	BMI	$N = 1$	128
\$90	BCC	$C = 0$	depends on entry state
\$B0	BCS	$C = 1$	depends on entry state
\$50	BVC	$V = 0$	depends on entry state
\$70	BVS	$V = 1$	depends on entry state

For a *complete* replicator (copies all 256 bytes without interruption), we need the branch taken for 255 consecutive iterations. Only **BNE** achieves this: it fails on exactly the one iteration where X wraps to 0, at which point all 256 bytes have been copied.

BPL and **BMI** fail on half the iterations, so the loop copies at most a few bytes before falling through. **BEQ** almost never branches. The carry- and overflow-dependent branches (**BCC**, **BCS**, **BVC**, **BVS**) depend on prior state: **INX/DEX** does not modify C or V , so if C was clear on entry, **BCC** always branches (all 256 iterations, including the wrap), and if C was set, **BCC** never branches.

In a randomly initialized cell, the carry flag has probability $1/2$ of being in either state. Therefore:

- **BCC** (\$90): works if $C = 0$ on entry (probability $1/2$).
- **BCS** (\$B0): works if $C = 1$ on entry (probability $1/2$).
- **BVC** (\$50): works if $V = 0$ on entry (probability $1/2$).
- **BVS** (\$70): works if $V = 1$ on entry (probability $1/2$).

For **BNE**: the branch is not taken when X wraps to 0, but by then all 256 bytes have been copied, and in zero-initialized RAM the fall-through byte is \$00 (**BRK**), which resets PC to 0. So the replicator is functional.

For the flag-dependent branches, if the flag is in the correct state, the branch is taken on all 256 iterations (the loop never terminates naturally, but the scheduler eventually interrupts). The copy may or may not complete within one scheduler quantum, but statistically it will complete over multiple quanta since X is saved and restored.

Counting valid branch opcodes:

- Unconditionally reliable: **BNE** (\$D0) — 1 opcode.
- Reliable with probability $1/2$: **BCC** (\$90), **BCS** (\$B0), **BVC** (\$50), **BVS** (\$70) — 4 opcodes.
- Unreliable: **BPL** (\$10), **BMI** (\$30), **BEQ** (\$F0) — excluded.

For a conservative count, we take only **BNE**: 1 valid opcode. For a generous count including the flag-dependent branches with their $1/2$ probability weight:

$$\text{effective count} = 1 + 4 \times \frac{1}{2} = 3.$$

Byte 6 constraint (generous): $\log_2(256/3) \approx 6.415$ bits.

Byte 7 constraint: must be \$F8 exactly. 8 bits.

Total for branch loop (generous): ≈ 14.415 bits over 2 bytes.

4.7.3 BRK variant (bytes 6+)

If byte 6 is \$00 (**BRK**), the scheduler resets PC to 0. The **BRK** instruction reads one operand byte (byte 7); in zero-initialized RAM, this is already \$00.

The critical question is whether byte 6 must be exactly \$00, or whether it is “free” because it falls in the zero-initialized background.

Case 1: Explicit BRK. We require byte 6 = \$00. Since the remaining bytes (7+) are unconstrained (they are zero by initialization, and the **BRK** operand is ignored beyond being read), we have 1 constrained byte. Constraint: 8 bits for byte 6.

Case 2: Implicit BRK. If we assume zero initialization, then any program of length ≤ 6 bytes is followed by \$00 bytes. A 6-byte replicator (bytes 0–5: B5 00 9D 00 04 E8) is followed by \$00 at byte 6 “for free.” Constraint: 0 additional bits.

In this paper, we treat both cases. For random initialization (all bytes uniform), byte 6 must be \$00 explicitly (Case 1). For zero-initialized memory, only 6 bytes are constrained (Case 2).

5 Summary of Byte Constraints

We present the effective information content for each variant.

5.1 JMP variant (9 bytes, random initialization)

Byte	Content	Valid values	Probability	Bits
0	LDA zp ,X opcode	1 (/256)	2^{-8}	8.000
1	Zero-page base addr	1 (/256)	2^{-8}	8.000
2	STA abs ,X opcode	1 (/256)	2^{-8}	8.000
3	Dest addr low byte	1 (/256)	2^{-8}	8.000
4	Dest addr high byte	48 (/256)	48/256	2.415
5	INX or DEX	2 (/256)	2/256	7.000
6	JMP opcode	1 (/256)	2^{-8}	8.000
7	Jump addr low byte	1 (/256)	2^{-8}	8.000
8	Jump addr high byte	1 (/256)	2^{-8}	8.000
Total				65.415

$$P_{\text{rep}}^{(\text{JMP})} = \frac{1 \cdot 1 \cdot 1 \cdot 1 \cdot 48 \cdot 2 \cdot 1 \cdot 1 \cdot 1}{256^9} = \frac{96}{256^9} = \frac{96}{2^{72}} = \frac{3}{2^{67}} \approx 2^{-65.415}.$$

5.2 Branch variant (8 bytes, random initialization)

Byte	Content	Valid values	Probability	Bits
0	LDA zp ,X opcode	1 (/256)	2^{-8}	8.000
1	Zero-page base addr	1 (/256)	2^{-8}	8.000
2	STA abs ,X opcode	1 (/256)	2^{-8}	8.000
3	Dest addr low byte	1 (/256)	2^{-8}	8.000
4	Dest addr high byte	48 (/256)	48/256	2.415
5	INX or DEX	2 (/256)	2/256	7.000
6	Branch opcode	1-3 (/256)	1/256-3/256	6.415-8
7	Branch offset	1 (/256)	2^{-8}	8.000
Total (conservative, BNE only)				57.415
Total (generous, BNE + flag-dependent)				55.830

Conservative (BNE only):

$$P_{\text{rep}}^{(\text{BNE})} = \frac{1 \cdot 1 \cdot 1 \cdot 1 \cdot 48 \cdot 2 \cdot 1 \cdot 1}{256^8} = \frac{96}{2^{64}} = \frac{3}{2^{59}} \approx 2^{-57.415}.$$

Generous (effective 3 branch opcodes):

$$P_{\text{rep}}^{(\text{branch})} = \frac{1 \cdot 1 \cdot 1 \cdot 1 \cdot 48 \cdot 2 \cdot 3 \cdot 1}{256^8} = \frac{288}{2^{64}} = \frac{9}{2^{59}} \approx 2^{-55.830}.$$

5.3 BRK variant (random initialization)

Byte	Content	Valid values	Probability	Bits
0	LDA zp ,X opcode	1 (/256)	2^{-8}	8.000
1	Zero-page base addr	1 (/256)	2^{-8}	8.000
2	STA abs ,X opcode	1 (/256)	2^{-8}	8.000
3	Dest addr low byte	1 (/256)	2^{-8}	8.000
4	Dest addr high byte	48 (/256)	48/256	2.415
5	INX or DEX	2 (/256)	2/256	7.000
6	BRK opcode	1 (/256)	2^{-8}	8.000
Total (7 bytes, random init)				49.415

$$P_{\text{rep}}^{(\text{BRK},7)} = \frac{96}{256^7} = \frac{96}{2^{56}} = \frac{3}{2^{51}} \approx 2^{-49.415}.$$

With zero initialization (byte 6 is free):

$$P_{\text{rep}}^{(\text{BRK},6)} = \frac{96}{256^6} = \frac{96}{2^{48}} = \frac{3}{2^{43}} \approx 2^{-41.415}.$$

5.4 Combined probability (union over all variants)

A random byte sequence is a valid replicator if it matches *any* of the variants. Since the variants have disjoint byte patterns at position 6 (\$4C for JMP, \$D0/\$90/\$B0/\$50/\$70 for branches, \$00 for BRK), their probabilities are additive up to the shared prefix:

$$\begin{aligned} P_{\text{rep}}^{(\text{total})} &= P_{\text{rep}}^{(\text{JMP})} + P_{\text{rep}}^{(\text{branch})} + P_{\text{rep}}^{(\text{BRK},7)} \\ &= \frac{96}{256^9} + \frac{288}{256^8} + \frac{96}{256^7} \\ &= \frac{96}{2^{72}} + \frac{288}{2^{64}} + \frac{96}{2^{56}} \end{aligned} \tag{1}$$

The BRK term dominates:

$$P_{\text{rep}}^{(\text{total})} \approx \frac{96}{2^{56}} = \frac{3}{2^{51}} \approx 2^{-49.415}.$$

The JMP and branch terms are negligible by comparison (smaller by factors of 2^{-16} and 2^{-8} respectively).

6 NOP Insertion Tolerance

6.1 Setup

The replicator’s four instructions (load, store, increment, loop) are independent in the sense that inserting “no-operation” instructions between them does not change the computation. A NOP does not modify registers A, X, Y, or the status flags relevant to the branch condition.

We allow NOP insertions at $K = 4$ positions:

1. Before the load (leading NOPs, j_0 bytes)
2. Between load and store (j_1 bytes)
3. Between store and increment (j_2 bytes)
4. Between increment and loop (j_3 bytes)

Let $J = j_0 + j_1 + j_2 + j_3$ be the total number of NOP bytes inserted. The resulting program has length $N_0 + J$ bytes, where N_0 is the base replicator length.

6.2 Counting NOP opcodes

The official 6502 NOP is \$EA (1 byte, 2 cycles). The 6502 also has several undocumented single-byte NOPs that are functionally equivalent (no register or flag side effects):

$$\text{\$1A, \$3A, \$5A, \$7A, \$DA, \$FA.}$$

This gives $n_{\text{NOP}} = 7$ single-byte NOP opcodes.

There are also multi-byte undocumented NOPs (2-byte and 3-byte), but these complicate the analysis significantly because their operand bytes add additional degrees of freedom. We defer multi-byte NOPs to Section 6.4.

6.3 Generating function for NOP insertions

For each insertion point, the number of single-byte NOP bytes inserted is $j_i \geq 0$, and each such byte must be one of n_{NOP} values. The probability of j consecutive NOP bytes at a single insertion point is

$$\left(\frac{n_{\text{NOP}}}{256}\right)^j.$$

The total probability contribution from NOP-padded replicators is:

$$P_{\text{rep}}^{(\text{NOP})} = P_{\text{core}} \cdot \sum_{j_0=0}^{\infty} \sum_{j_1=0}^{\infty} \sum_{j_2=0}^{\infty} \sum_{j_3=0}^{\infty} \left(\frac{n_{\text{NOP}}}{256}\right)^{j_0+j_1+j_2+j_3} \cdot A(j_0) \quad (2)$$

where P_{core} is the probability of the core instruction bytes (excluding the loop-back), and $A(j_0)$ is a correction factor for the loop instruction's operand, which depends on the total program length and hence on the NOP counts.

Branch variant correction. For the branch variant, the branch offset depends on the total program length $L = N_0 + J$. From byte position $L - 2$ (the branch opcode), the required offset to reach byte 0 is $-(L)$, encoded as a signed byte. This is valid only if $L \leq 128$ (the maximum backward branch range is -128). For $L \leq 128$, the offset byte is uniquely determined ($256 - L$). So $A(j_0) = 1$ for all valid J , and the branch offset is always exactly 1 value.

However, there is a subtlety for leading NOPs ($j_0 > 0$): the loop branches back to byte 0, which is a NOP, not the load instruction. The replicator still works because the NOPs are re-executed each iteration. The branch offset must target byte 0, not the first non-NOP instruction.

For the branch variant with leading NOPs, the branch offset is $-L = -(N_0 + J)$. This is valid as a signed byte when $L \leq 128$, i.e., $J \leq 128 - N_0 = 120$ (for $N_0 = 8$). Each valid offset is one specific byte value. So $A(j_0) = 1$ and the constraint on the offset byte remains 8 bits regardless of J .

JMP variant correction. The JMP \$0000 operand is always \$00 00 regardless of program length. So $A(j_0) = 1$ trivially.

BRK variant correction. BRK resets PC to 0 regardless of where it appears. $A(j_0) = 1$ trivially.

Therefore, in all cases $A(j_0) = 1$, and Equation (2) simplifies:

$$\begin{aligned} P_{\text{rep}}^{(\text{NOP})} &= P_{\text{core}} \cdot \left(\sum_{j=0}^{\infty} \left(\frac{n_{\text{NOP}}}{256}\right)^j \right)^K \\ &= P_{\text{core}} \cdot \left(\frac{1}{1 - n_{\text{NOP}}/256} \right)^K \\ &= P_{\text{core}} \cdot \left(\frac{256}{256 - n_{\text{NOP}}} \right)^K. \end{aligned} \quad (3)$$

Proposition 2 (NOP correction factor). *With $n_{\text{NOP}} = 7$ single-byte NOP opcodes and $K = 4$ insertion points, the NOP-tolerance multiplicative factor is*

$$F_{\text{NOP}} = \left(\frac{256}{249}\right)^4 \approx 1.1174.$$

This increases the replicator probability by a factor of ≈ 1.12 , reducing the effective information content by

$$\Delta B = \log_2 F_{\text{NOP}} = 4 \log_2 \left(\frac{256}{249}\right) \approx 0.160 \text{ bits.}$$

Proof. The geometric series $\sum_{j=0}^{\infty} x^j = (1 - x)^{-1}$ for $|x| < 1$. With $x = n_{\text{NOP}}/256 = 7/256 \approx 0.02734$, the series converges and equals $256/249$. Raising to the $K = 4$ th power:

$$F_{\text{NOP}} = \left(\frac{256}{249}\right)^4.$$

Numerically: $256/249 \approx 1.02811$, so $F_{\text{NOP}} \approx 1.02811^4 \approx 1.1174$. And $\log_2(1.1174) \approx 0.160$. □

6.4 Multi-byte NOP instructions

The 6502 has undocumented 2-byte NOPs (opcode + 1 operand byte) and 3-byte NOPs (opcode + 2 operand bytes) that have no side effects. These provide additional NOP-equivalent sequences:

- **2-byte NOPs:** Opcodes \$80, \$82, \$C2, \$E2, \$04, \$14, \$34, \$44, \$54, \$64, \$74, \$D4, \$F4. Each takes 1 operand byte that can be anything (256 values). This gives $n_2 \approx 13$ distinct 2-byte NOP opcodes. Probability per 2-byte NOP: $(n_2/256) \cdot 1 = n_2/256$.
- **3-byte NOPs:** Opcodes \$0C, \$1C, \$3C, \$5C, \$7C, \$DC, \$FC. Each takes 2 operand bytes. This gives $n_3 = 7$ distinct 3-byte NOP opcodes. Probability per 3-byte NOP: $(n_3/256) \cdot 1 \cdot 1 = n_3/256$.

Wait — a 2-byte NOP inserted at a gap adds 2 bytes to the program length, with probability $(n_2/256) \cdot (256/256) = n_2/256$ for the pair (the operand byte is unconstrained, contributing a factor of $256/256 = 1$). Similarly, a 3-byte NOP adds 3 bytes with probability $n_3/256$.

Actually, we must be more careful. Each NOP “slot” can hold a sequence of NOP instructions of various lengths. Let p_k be the probability that a specific k -byte NOP instruction appears, consuming k bytes from the random stream:

$$\begin{aligned} p_1 &= n_1/256 = 7/256, \\ p_2 &= n_2/256 = 13/256, \quad (\text{operand byte is free}) \\ p_3 &= n_3/256 = 7/256. \quad (\text{operand bytes are free}) \end{aligned}$$

Note that for multi-byte NOPs, we “consume” additional random bytes from the stream, but those bytes are unconstrained (any value is a valid operand). The probability cost is only the opcode byte.

At each insertion point, the content is a sequence of NOP instructions. Let $G(z)$ be the generating function where z tracks the number of random bytes consumed. A single NOP instruction consuming k bytes contributes $p_k z^k$ to the generating function. The generating function for zero or more NOPs at one insertion point is:

$$H(z) = \sum_{m=0}^{\infty} \left(\sum_{k=1}^3 p_k z^k \right)^m = \frac{1}{1 - (p_1 z + p_2 z^2 + p_3 z^3)}.$$

The total probability of a NOP-padded replicator is obtained by evaluating $H(z)$ at $z = 1/256$ for each consumed byte... but wait, this is not quite right. The generating function approach needs to account for the fact that each random byte costs a factor of $1/256$ in probability.

Let us re-derive. Suppose a NOP insertion point consumes L bytes total. The probability that L specific random bytes form a valid sequence of NOP instructions is:

$$Q(L) = \sum_{\text{NOP sequences of length } L \text{ instructions}} \prod \frac{1}{256^{k_i}}$$

where k_i is the length of the i th NOP instruction.

For a k -byte NOP instruction, the opcode must be one of n_k values (probability $n_k/256$) and the remaining $k - 1$ operand bytes are free (probability 1 each). So the probability cost per instruction is $n_k/256$, independent of instruction length. This is because the $k - 1$ operand bytes are unconstrained — they consume $k - 1$ bytes from the random stream but impose no constraint.

Wait, that is wrong. Each byte in the random 1024-byte cell is drawn uniformly. A k -byte NOP instruction “uses up” k bytes. The first byte must be a valid NOP opcode ($n_k/256$ probability). The remaining $k - 1$ bytes can be anything (1 probability each). But these $k - 1$ bytes are still random bytes that could have been something else. The total probability that bytes $i, i + 1, \dots, i + k - 1$ form a specific k -byte NOP is $(n_k/256) \cdot 1^{k-1} = n_k/256$.

But the remaining $k - 1$ bytes are “consumed” — they cannot be part of another NOP instruction or the replicator core. This consumption is already accounted for by the fact that the replicator has a specific total length. Let L_{total} be the total program length (core + all NOP bytes). Then $P_{\text{rep}} = 256^{-L_{\text{total}}} \times (\text{number of valid patterns of that length})$. Actually, since some bytes within multi-byte NOPs are unconstrained, the probability per byte is not uniformly $1/256$.

Let me restart with a cleaner framework.

Definition 2 (NOP insertion probability). *Fix the core replicator bytes. A NOP-padded replicator of total length L is a sequence of L bytes that consists of the core replicator instructions interspersed with NOP instruction sequences at the K designated insertion points, such that the total byte count is L . The probability of a random L -byte prefix matching such a pattern is*

$$P(L) = P_{\text{core}} \cdot \prod_{i=1}^K q_i(\ell_i)$$

where ℓ_i is the number of NOP bytes at insertion point i (with $\sum_i \ell_i = L - N_0$), P_{core} accounts for the constrained core bytes, and $q_i(\ell)$ is the probability that ℓ consecutive random bytes form a valid sequence of NOP instructions.

The function $q(\ell)$ (same for all insertion points) satisfies:

- $q(0) = 1$ (no NOP bytes).
- For $\ell \geq 1$: the first byte starts a NOP instruction of length $k \in \{1, 2, 3\}$, with $k \leq \ell$. The probability that the first byte is a valid k -byte NOP opcode is $n_k/256$. The next $k - 1$ bytes are unconstrained (free). The remaining $\ell - k$ bytes must also form a valid NOP sequence.

$$q(\ell) = \sum_{k=1}^{\min(\ell, 3)} \frac{n_k}{256} \cdot q(\ell - k) \quad \text{for } \ell \geq 1.$$

This is a linear recurrence. The generating function is $Q(z) = \sum_{\ell=0}^{\infty} q(\ell)z^\ell$, satisfying:

$$Q(z) = 1 + \left(\frac{n_1}{256}z + \frac{n_2}{256}z^2 + \frac{n_3}{256}z^3 \right) Q(z)$$

so

$$Q(z) = \frac{1}{1 - \frac{1}{256}(n_1z + n_2z^2 + n_3z^3)}. \quad (4)$$

The total probability summing over all possible NOP insertion lengths is:

$$\begin{aligned} P_{\text{rep}}^{(\text{NOP})} &= P_{\text{core}} \cdot \sum_{\ell_1, \dots, \ell_K \geq 0} \prod_{i=1}^K q(\ell_i) \\ &= P_{\text{core}} \cdot \left(\sum_{\ell=0}^{\infty} q(\ell) \right)^K \\ &= P_{\text{core}} \cdot Q(1)^K. \end{aligned} \quad (5)$$

Proposition 3 (NOP factor with multi-byte NOPs). *With $n_1 = 7$, $n_2 = 13$, $n_3 = 7$ and $K = 4$:*

$$Q(1) = \frac{1}{1 - (7 + 13 + 7)/256} = \frac{256}{256 - 27} = \frac{256}{229} \approx 1.1179.$$

The NOP correction factor is

$$F_{\text{NOP}} = Q(1)^4 = \left(\frac{256}{229} \right)^4 \approx 1.5608.$$

This reduces the effective information content by

$$\Delta B = \log_2(1.5608) \approx 0.642 \text{ bits}.$$

Proof. From Equation (4) with $z = 1$:

$$Q(1) = \frac{1}{1 - (n_1 + n_2 + n_3)/256} = \frac{256}{256 - 27} = \frac{256}{229}.$$

Numerically: $256/229 \approx 1.11790$, and $(256/229)^4 \approx 1.5608$. Then $\log_2(1.5608) \approx 0.6424$. $\square$

Remark 3. *The multi-byte NOP generating function Equation (4) correctly accounts for the fact that multi-byte NOP operand bytes are unconstrained: $q(\ell)$ gives the probability that ℓ consecutive random bytes form a valid NOP sequence, where each random byte independently has probability $1/256$ of being any specific value. The “free” operand bytes contribute factors of $256/256 = 1$ to the probability, which is why $n_k/256$ (not $n_k/256^k$) appears in the recurrence.*

Remark 4 (Truncation effects). *The generating function $Q(1)$ sums over $\ell = 0, 1, 2, \dots, \infty$. In practice, the NOP sequence cannot exceed $1024 - N_0$ bytes. Since $q(\ell)$ decays exponentially as $(27/256)^\ell \approx 0.1055^\ell$, the tail beyond $\ell = 10$ contributes less than $(0.1055)^{10} \approx 1.5 \times 10^{-10}$ to $Q(1)$. The infinite sum is an excellent approximation.*

7 Additional Functional Equivalences

7.1 Starting position within the cell

The replicator must start at the entry point, which is byte 0 after each scheduler interrupt. Leading NOPs (the j_0 insertion point) are the only way to shift the “real” code away from byte 0 while maintaining functionality. This is already accounted for in Section 6.

7.2 Alternative copy directions

Using DEX (\$CA) instead of INX (\$E8) copies bytes in the order $x_0, x_0 - 1, \dots, 0, 255, \dots, x_0 + 1$. All 256 bytes are still copied. This is already counted (2 valid values at byte 5).

7.3 Copy to different cells

As analyzed in Section 4.5, any of the 48 neighbor cells is a valid target. This is already counted (48 valid values at byte 4).

7.4 Partial copies and multi-quantum replication

The analysis above assumes the replicator completes a full 256-byte copy within a single scheduler quantum. In practice, the replicator may be interrupted partway through and resume in a later quantum (to a different, randomly chosen neighbor). This “scattered copy” mode is less reliable but could still result in eventual replication if the same neighbor is targeted enough times. We do not quantify this here; our analysis is conservative in requiring a single-quantum complete copy.

7.5 Self-modifying or longer replicators

Longer programs could implement replication through entirely different mechanisms (e.g., using a subroutine, using indirect addressing, using self-modifying code). These are vastly more complex and correspondingly less probable under random initialization. We do not attempt to enumerate them.

8 Effective Information Content

Combining all relaxations from Sections 4 and 6, we compute the effective information content B_{eff} , defined as $B_{\text{eff}} = -\log_2 P_{\text{rep}}$.

8.1 BRK variant, random initialization (most favorable)

Core probability (7 bytes):

$$P_{\text{core}}^{(\text{BRK},7)} = \frac{1 \cdot 1 \cdot 1 \cdot 1 \cdot 48 \cdot 2 \cdot 1}{256^7} = \frac{96}{256} = \frac{3}{2^{51}}.$$

With NOP correction (Proposition 3):

$$P_{\text{rep}}^{(\text{BRK}, \text{NOP})} = \frac{3}{2^{51}} \times \left(\frac{256}{229} \right)^4 \approx \frac{3}{2^{51}} \times 1.5608.$$

$$B_{\text{eff}}^{(\text{BRK}, \text{NOP})} = 51 - \log_2 3 - \log_2 1.5608 \approx 51 - 1.585 - 0.642 \approx 48.773 \text{ bits.}$$

8.2 Branch variant, random initialization, generous opcode count

Core probability (8 bytes, 3 effective branch opcodes):

$$P_{\text{core}}^{(\text{branch})} = \frac{1 \cdot 1 \cdot 1 \cdot 1 \cdot 48 \cdot 2 \cdot 3 \cdot 1}{256^8} = \frac{288}{2^{64}} = \frac{9}{2^{59}}.$$

With NOP correction:

$$P_{\text{rep}}^{(\text{branch}, \text{NOP})} = \frac{9}{2^{59}} \times \left(\frac{256}{229} \right)^4.$$

$$B_{\text{eff}}^{(\text{branch}, \text{NOP})} = 59 - \log_2 9 - \log_2 1.5608 \approx 59 - 3.170 - 0.642 \approx 55.188 \text{ bits.}$$

8.3 Union over all variants with NOP correction

The total probability is dominated by the BRK variant (shortest):

$$P_{\text{rep}}^{(\text{total})} \approx P_{\text{rep}}^{(\text{BRK}, \text{NOP})} \approx 2^{-48.773}.$$

Theorem 1 (Effective information content). *The effective information content of the minimal 6502 self-replicator, accounting for alternative target cells (48 neighbors), bidirectional index stepping (INX/DEX), and NOP insertion tolerance (7 single-byte + 13 two-byte + 7 three-byte undocumented NOP opcodes at 4 insertion points), is*

$$\boxed{B_{\text{eff}} \approx 48.8 \text{ bits}}$$

for the BRK variant under random initialization, or $B_{\text{eff}} \approx 40.8$ bits under zero initialization (where the trailing BRK is free).

9 Primordial Soup: Probability of Spontaneous Emergence

9.1 Expected number of replicators on a random board

Consider a board of $B \times B$ cells, each with $M = 1024$ bytes drawn uniformly at random. The replicator must start at byte 0 (the execution entry point), so each cell has exactly one “trial” for the replicator pattern.

Proposition 4. *The expected number of replicators on a $B \times B$ board is*

$$\mathbb{E}[R] = B^2 \cdot P_{\text{rep}}.$$

For the standard 256×256 board ($B^2 = 65,536 = 2^{16}$):

$$\mathbb{E}[R] = 2^{16} \cdot 2^{-B_{\text{eff}}}.$$

Scenario	B_{eff}	$\mathbb{E}[R]$ on 256^2 board
BRK, random init, with NOPs	48.8	$2^{-32.8} \approx 1.4 \times 10^{-10}$
BRK, zero init, with NOPs	40.8	$2^{-24.8} \approx 3.4 \times 10^{-8}$
Branch, random init, with NOPs	55.2	$2^{-39.2} \approx 1.7 \times 10^{-12}$
JMP, random init, with NOPs	64.8	$2^{-48.8} \approx 2.8 \times 10^{-15}$

In all cases, the expected number of replicators on a standard board is astronomically small. Even the most favorable scenario (BRK variant, zero initialization) gives $\mathbb{E}[R] \approx 10^{-8}$.

9.2 Threshold board size

For $\mathbb{E}[R] = 1$, we need $B^2 = 2^{B_{\text{eff}}}$, i.e., $B = 2^{B_{\text{eff}}/2}$.

Scenario	B_{eff}	Board side B for $\mathbb{E}[R] = 1$
BRK, zero init, with NOPs	40.8	$2^{20.4} \approx 1.4 \times 10^6$
BRK, random init, with NOPs	48.8	$2^{24.4} \approx 2.2 \times 10^7$
Branch, random init, with NOPs	55.2	$2^{27.6} \approx 2.0 \times 10^8$
JMP, random init, with NOPs	64.8	$2^{32.4} \approx 5.5 \times 10^9$

9.3 Memory requirements

Each cell requires 1024 bytes = 1 KiB. A board of side B requires $B^2 \times 1024$ bytes = B^2 KiB.

Scenario	Board side B	Memory
BRK, zero init	1.4×10^6	≈ 1.8 TiB
BRK, random init	2.2×10^7	≈ 450 TiB
Branch, random init	2.0×10^8	≈ 37 PiB
JMP, random init	5.5×10^9	≈ 28 EiB

Even the most favorable scenario requires ~ 1.8 TiB of memory for a board where the expected number of spontaneous replicators is 1. This is within reach of modern hardware but represents a board of $1.4 \times 10^6 \times 1.4 \times 10^6 \approx 2 \times 10^{12}$ cells — far larger than the standard 256×256 board.

9.4 Comparison with biological estimates

For context, estimates of the probability of spontaneous emergence of a self-replicating RNA molecule range from 10^{-40} to 10^{-100} per random nucleotide sequence of appropriate length. Our estimate of $\sim 2^{-49} \approx 10^{-14.7}$ for the BRK variant reflects the relative simplicity of the 6502 instruction set compared to biochemistry: a 6502 replicator requires only ~ 49 bits of specific information, whereas even the simplest ribozyme replicators require hundreds of specific nucleotides.

10 Sensitivity Analysis

10.1 Undocumented NOP counts

The counts $n_1 = 7$, $n_2 = 13$, $n_3 = 7$ depend on the specific 6502 variant emulated. The NMOS 6502 has more undocumented NOPs than the CMOS 65C02. The Sfotty emulator used in 6502life may implement a different subset. The NOP correction factor is relatively insensitive to these counts:

(n_1, n_2, n_3)	$Q(1)$	$F_{\text{NOP}} = Q(1)^4$
(1, 0, 0)	$256/255 = 1.0039$	1.0157
(7, 0, 0)	$256/249 = 1.0281$	1.1174
(7, 13, 7)	$256/229 = 1.1179$	1.5608
(7, 20, 7)	$256/222 = 1.1532$	1.7695

The NOP correction ranges from negligible (+0.02 bits) to modest (+0.82 bits), confirming that it is a minor effect.

10.2 Functional but imperfect replicators

Our analysis counts only “perfect” byte-level replicators that produce an exact copy of page 0. A weaker definition might allow replicators that copy enough code for the child to also replicate, even if some non-code bytes are incorrect. This could significantly increase the probability if the replicator were robust to background byte variations, but the minimal replicators we study copy the entire page and are therefore sensitive to the background (the child’s page 0 is an exact copy of the parent’s page 0, including any random garbage in non-code bytes).

10.3 Replicator as a substring

Our analysis assumes the replicator starts at byte 0 (the entry point). An alternative scenario: the replicator code exists somewhere in the 1024-byte cell (not necessarily at byte 0), and some sequence of instructions at byte 0 eventually transfers control to it. This is much harder to quantify, as it depends on the probability that random code produces a jump to the correct offset. We conjecture this increases P_{rep} by at most a factor of ~ 1024 (one extra trial per byte position), reducing B_{eff} by at most ~ 10 bits. But the probability that random prefix code successfully reaches the replicator without crashing or looping is very low, likely offsetting this gain.

11 Dominance Time and Lottery Economics

11.1 Replicator dominance time

A replicator seeded at one cell spreads via the checkerboard pairing scheme. Each half-pass, the replicator may copy to its paired neighbor if it is the active cell in the pair (probability $\frac{1}{2}$) and the quantum budget suffices for the copy loop (which it does for any budget $\gtrsim 100$ cycles, essentially always).

The spreading wavefront advances at most 1 cell per half-pass in one cardinal direction. The two half-passes in each round use orthogonal tilings (horizontal and vertical), so the wavefront can advance 1 cell horizontally and 1 cell vertically per round. Including the random offset bits (which shift the tiling grid by 0 or 1), every cell boundary is crossed by some pairing within $O(1)$ rounds.

For an $L \times L$ board, the replicator must traverse at most L cells in each dimension. With a 50% activation probability per pair, the expected number of rounds to advance one cell is 2 (geometric with $p = \frac{1}{2}$). The expected dominance time is therefore:

$$D(L) \approx 2L \text{ rounds} = 4L \text{ half-passes (runPass calls)}.$$

Each half-pass gives each cell one quantum of $\mu = 4096$ mean cycles. In cycles per cell:

$$D_{\text{cyc}}(L) = D(L) \times \mu = 4L \times 4096 \approx 16,384 L.$$

For $L = 32$: $D_{\text{cyc}} \approx 524,288$ cycles/cell $\approx 0.5 \times 10^6$. At $C = 200,000$ cyc/cell/s, dominance takes ≈ 2.6 seconds.

Note this is a lower bound on the mean; the actual distribution has a tail from the random walk of the wavefront boundary. A more precise analysis using first-passage times of a biased 2D random walk gives $D(L) = \Theta(L)$ with moderate variance ($\sigma \sim \sqrt{L}$).

11.2 Throughput: boards tested per unit time

A player initializes a board with SHA-256 pseudorandom data and runs the simulation to detect whether a viable replicator emerges. Detection requires running long enough for a replicator (if present) to achieve observable dominance—say, colonizing $> 50\%$ of cells.

At rate C cycles/cell/s on an $L \times L$ board:

- Rounds per second: $C/(2\mu) \approx C/8192$.
- Time to dominance: $D(L)/(C/2\mu) = 4L \cdot 2\mu/C = 8\mu L/C$ seconds.

- Boards per second: $C/(8\mu L) \approx C/(32,768 L)$.

For $L = 32$, $C = 200,000$:

Time per board = $8 \times 4096 \times 32/200,000 \approx 5.2$ seconds,
Boards per hour ≈ 690 .

For $L = 16$, $C = 800,000$ (smaller board, faster per-cell rate since $C \propto L^{-2}$ on fixed hardware):

Time per board ≈ 0.65 seconds,
Boards per hour $\approx 5,500$.

11.3 Lottery ticket probability

Each cell in a randomly initialized board is an independent trial for containing a replicator at position 0. With L^2 cells, the expected number of replicators per board is:

$$\mathbb{E}[\text{replicators per board}] = L^2 \cdot 2^{-B_{\text{eff}}}.$$

For a player seeking at least one replicator, the expected number of boards until success is:

$$N_{\text{boards}} = \frac{1}{L^2 \cdot 2^{-B_{\text{eff}}}} = \frac{2^{B_{\text{eff}}}}{L^2}.$$

Random initialization ($B_{\text{eff}} \approx 55$ bits). For $L = 32$: $N_{\text{boards}} = 2^{55}/1024 = 2^{45} \approx 3.5 \times 10^{13}$. At 690 boards/hour, this requires $\sim 5 \times 10^{10}$ hours ≈ 5.8 million years. **Infeasible.**

Zero initialization ($B_{\text{eff}} \approx 41$ bits). If RAM is zero-initialized and only the first few bytes are randomized (not all 1024), the BRK variant has $B_{\text{eff}} \approx 41$: $N_{\text{boards}} = 2^{41}/1024 = 2^{31} \approx 2.1 \times 10^9$. At 690 boards/hour: $\sim 3 \times 10^6$ hours ≈ 350 years. **Still infeasible for a single player, but conceivable for a network.**

BRK-copy primitive ($B_{\text{eff}} \approx 12.4$ bits). If the VM's BRK-copy operation is enabled (operands 49–96 trigger a built-in noisy cell copy), a “replicator” is simply BRK followed by one of 48 valid operands. This is 2 constrained bytes with 48/256 probability for the second: $P = 48/65536 \approx 2^{-10.4}$ per cell. For a 32×32 board: $\mathbb{E} = 1024 \times 2^{-10.4} \approx 0.75$ replicators per board. With ≈ 1.3 boards, a player expects success. **Trivially achievable**—but BRK-copy replicators are noisy and may not sustain themselves against mutation.

Summary of lottery economics.

Scenario	B_{eff}	$N_{\text{boards}} (L=32)$	Time (690 brd/hr)	Feasible?
JMP, random	65	3×10^{17}	5×10^{10} yr	No
Branch, random	55	3×10^{13}	5×10^6 yr	No
BRK, zero-init	41	2×10^9	350 yr	Network
BRK-copy primitive	10	1.3	7 sec	Yes

The stark conclusion: programmatic self-replicators (byte-copy loops) are astronomically unlikely to arise by chance. The VM's built-in BRK-copy mechanism, if enabled, is the only plausible path to spontaneous replication—but it produces noisy, fragile replicators that must evolve error tolerance to persist.

12 Conclusion

	Strict P_{base}	Relaxed P_{rep}	B_{eff}
JMP (9 bytes)	2^{-72}	$2^{-64.8}$	64.8
Branch (8 bytes)	2^{-64}	$2^{-55.2}$	55.2
BRK, random init (8 bytes) ¹	2^{-64}	$2^{-56.8}$	56.8
BRK, zero init (6 bytes)	2^{-48}	$2^{-40.8}$	40.8

The minimal self-replicator in the 6502life VM has an effective information content of approximately 41–57 bits, depending on the assumed initial conditions. Spontaneous emergence on a standard 256×256 board is negligible ($\sim 10^{-8}$ to 10^{-15} expected replicators). A board of side $\sim 10^6$ (requiring ~ 1.8 TiB) would be needed for spontaneous replication to be expected under the most favorable assumptions.

The dominant relaxations are:

1. **Target cell flexibility** (byte 4): 48 valid neighbors instead of 1, saving ≈ 5.6 bits.
2. **Bidirectional stepping** (byte 5): **INX** or **DEX**, saving 1 bit.
3. **NOP insertion tolerance**: saving ≈ 0.6 bits (minor effect due to the low probability of NOP opcodes).

The remaining bytes (load opcode, base addresses, store opcode, loop mechanism) are tightly constrained with no or very few alternatives.

References

- [1] 6502life.com: A virtual 256×256 grid of interconnected 6502 CPUs simulating cellular automata. <https://6502life.com>, 2024–2026.

¹BRK operand byte (byte 7) must be \$00 under random initialization; under zero-init it is free.